



OSS コンプライアンス実務

この資料は、各企業が OSS（オープンソースソフトウェア）のサプライチェーンリスクマネジメントを行う際に、ライセンスの調査、ライセンス義務の履行、SBOM、セキュリティリスク対応を習得する上での参考教材です。各社で教育コンテンツを作成する際の材料として自由に複製、改変してご活用ください。

本資料は、[Creative Commons CC0 1.0 Universal](https://creativecommons.org/licenses/by/4.0/) ライセンスの下でリリースされており、複製、改変、配布にあたっての制限はありません。

【作成元：[OpenChain Japan Work Group education sg](https://openchain.jp/education/)】

Disclaimer（免責事項）

- 本資料は、日本国内で OSS を活用する場合を前提としており、education sg のメンバの経験に基づいて説明を記載しています。厳密な法解釈や海外での利用など、判断に迷う場合は専門家にご相談ください。
- 本資料に記載した内容については、作成者、提供元は一切の責任を負いませんので、ご承知のうえご利用ください。
- 本資料は、OpenChain の仕様書準拠や認定取得を保証するものではありません。

目次

1	OSS ライセンスの調査	3
1.1	導入 Q&A	3
1.2	ライセンスの調査	3
1.3	ライブラリ等のライセンスも確認	4
1.4	ライセンスの記載例	5
1.5	複数のライセンスが記載されているケース	7
1.6	Web 掲載のライセンスで誤認(バージョン確認)	9
1.7	OSS、ライセンスのリスト化	10
2	ライセンス義務の履行	11
2.1	導入 Q&A	11
2.2	ライセンス遵守作業の概要	11
2.3	必須文言（ライセンス文書等）の掲載	12
2.4	ソースコードの提供	14
2.5	製品に付帯する契約書（エンドユーザライセンス等）への追記	20
3	SBOM	21
3.1	ソフトウェアの構成を知る	21
3.2	SBOM を取得/作成する方法	21
3.3	SBOM で扱う項目とそのフォーマット	22
3.4	SBOM を管理するためのベストプラクティス	23
3.5	参考資料	24
4	OSS のセキュリティリスクへの対応	26
4.1	はじめに	26
4.2	OSS のセキュリティリスクの種類	26
4.3	OSS のセキュリティリスクを発見する方法	27
4.4	発見された脆弱性への対応	28
4.5	OSS のセキュリティリスク対策・予防	29
4.6	OSS セキュリティに役立つリソース（動向、レポート、ケーススタディ、オンラインコミュニティ、トレーニングなど）	30

図目次

図 1 OSS はバージョン毎のライセンス確認が必須	3
図 2 OSS に含まれる全ライブラリも要確認	4
図 3 ライセンスの記載例（アプリ）	6
図 4 ライセンスの記載例（Web サイト）	7
図 5 デュアルライセンスの例	8
図 6 トリプルライセンスの例	8
図 7 OSS か商用ライセンスかを選択	9
図 8 バージョンが変わるとライセンスも変わる例（Apache HTTP Server）	9
図 9 バージョンが変わるとライセンスも変わる例（jQuery）	10
図 10 レベル 1 の OSS ライセンス（MIT LICENSE）	13
図 11 謝辞の掲載を求めるライセンス例（Libjpeg License, https://jpegclub.org/reference/libjpeg-license/ ）	13
図 12 謝辞の記載例（Web ブラウザ「Firefox」、 about:license#other-notices ）	14
図 13 謝辞を広告媒体へ掲載するよう要求するライセンス条項の例（BSD 4-Clause Original (Old) License）	14
図 14 通信系組み込み製品における自社 Web サイトの製品紹介ページでの掲載例	14
図 15 ソースコードを同梱する三つの形態	16
図 16 ソースコード一を提供する旨の文書を同梱する	17
図 17 新型コロナウイルス接触確認アプリ「COCOA」のソースリポジトリ（Web 上で自身のソースコード一を公開）	19
図 18 新型コロナウイルス接触確認アプリ「COCOA」のソースリポジトリ（赤枠部分にて OSS「COVID-19Radar」のソースコード一を公開）	19
図 19 ソースコードの提供者はバイナリコードの提供者	20

表目次

表 1 ライセンス条件の制約の強さ（5レベル）	11
-------------------------------	----

1 OSS ライセンスの調査

1.1 導入 Q&A

ちょっと考えてみましょう。自社で OSS A を利用しています。インターネット上の Web サイトの記載は、『OSS A のライセンスはライセンス X』となっています。その OSS の Version 2 を配布する場合、ライセンス X で配布して良いでしょうか？

答えは、必ず利用する Version 2 のライセンスを確認して、対応するライセンスで配布する必要があります。

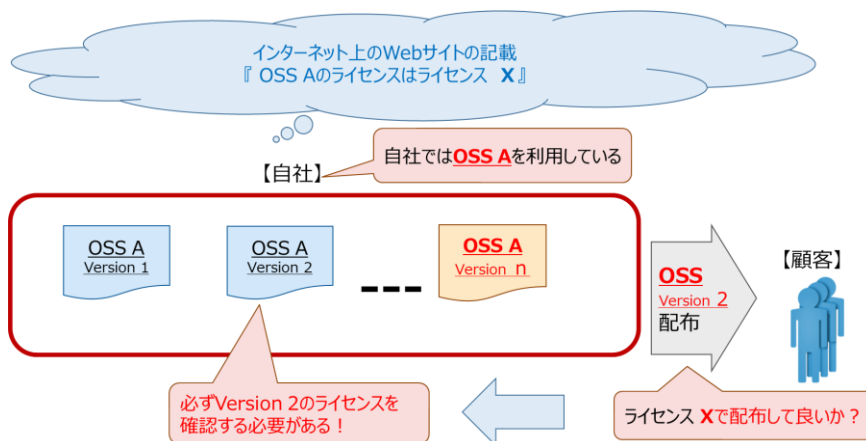


図 1 OSS はバージョン毎のライセンス確認が必須

下記の節で、ライセンスの調査に関する事項について確認していきましょう。

1.2 ライセンスの調査

OSS を利用する際には、ライセンスの調査を行い、利用時にライセンスを遵守する必要があります。OSS の選定時には、先ずダウンロードファイルの調査を行い、その OSS のライセンスを網羅的に調査する必要があります。

ライセンスの調査について、下記を実例として参考に紹介します。

(a) 先ず、対象となる OSS のダウンロードサイトを確認し、ダウンロードします。

ダウンロードする前に、「ダウンロードする時や利用するときはこのライセンスに従うように」等と記載がある場合は、そのライセンスに従う必要があるので、記載事項をライセンス情報として確認します。

(b)(a) でダウンロードしたファイルがアーカイブされたファイル(拡張子が

zip/jar/tar/tar.gz/tgz/xz/deb/rpm 等のファイル)である場合は、これを解凍します。

(c)(a) でダウンロードしたファイルが実行可能形式や Windows Installer 形式のファイル(拡張子が

exe,msi のファイル)である場合は、これをインストールします。

インストーラを起動し、表示されるライセンス本文やソフトウェア利用許諾契約等をライセンス情報として確認します。

- (d)ファイルの名称に下記の文字列が含まれるファイルを検索します。検索されたファイルにライセンス情報の記載があるかを確認します。

license/copying/copyright/eula/about.html/pom/manifest-readme/notice/legal/licence(※)等

- (e)ダウンロードファイルのすべてのファイルの中身に対して、下記の文字列が含まれているかを検索します。これらの文字列が含まれているファイルに、ライセンス情報の記載があるかを確認します。

license/epl/gpl/asl/artistic/legal/notice/third/licence(※)等

- (f)(b)でアーカイブされたファイルの場合は、アーカイブされたファイル中の META-INF フォルダ内の MANIFEST.MF/pom.xml 等の定義ファイルに、ライセンス情報が含まれていないかを確認します。

- (g)ダウンロードサイトと公式サイト、プロジェクトサイトやベンダーサイトが別の場合には、公式サイト、ベンダーサイト上にて下記の文字列を探し、ライセンス本文やライセンスへのリンクを調査します。

License/Licensing/Readme/Copying/Copyright/Eula/Legal/about.html/notice/Thirdparty/Thirdparties/Licence(※)

(※: licence は、license と同様の意味で使用される場合があります。)

上記の実施例の(a)～(f)の作業で、ソースコードファイル中のライセンス情報を網羅的に確認することが基本となります。(g)の作業は、あくまでも補助的な作業と考えた方がよいと思います。(a)～(f)の作業で調査したライセンスと、(g)の作業で調査したライセンスが異なる場合、前者のライセンスと後者のライセンスが異なる理由を明確にし、その理由に基づいてどちらを選択するかを判断し、いずれかを選択して使用した方がよいと思われる。

OSS のオブジェクトコードやロードモジュールを利用する場合でも、対応するソースコードをダウンロードして、ライセンスの調査を行う事が重要です。そこで、オブジェクトコードやロードモジュールのバージョンと、ソースコードのバージョンが一致している事を確認しておく必要があります。また、ロードモジュールには、ライブラリ等が追加されている場合もあります。追加されているライブラリ等のライセンスも確認が必要です。

上記の調査でライセンス情報が見つからない場合には、コミュニティの開発者に確認が必要でしょう。それでもライセンス情報が明確にならない場合は、その OSS は利用禁止にする必要があります。

1.3 ライブラリ等のライセンスも確認

近年、OSS も(特に OS やフレームワーク等で)大規模化が進んでおり、他の OSS のライブラリ等を含んでいるものが多々あります。そこで、1.2 項のライセンスの調査を確実にし、網羅的にその OSS に含まれて

いる他の OSS のライブラリ等とそれらのライセンスを調査する必要があります。

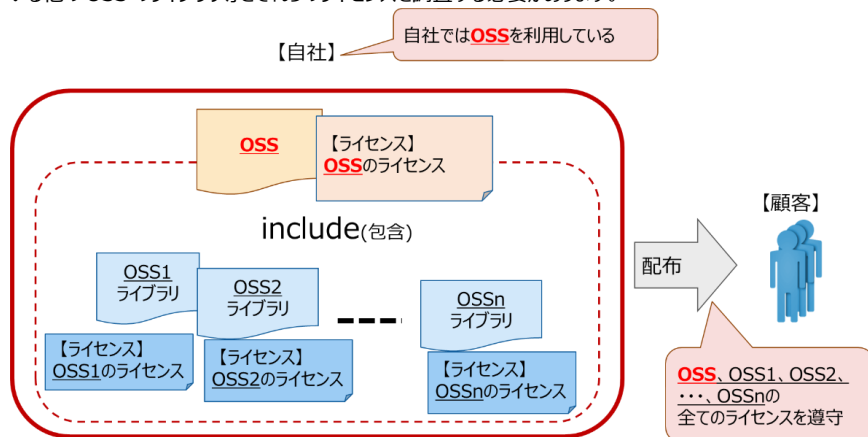


図 2 OSS に含まれる全ライブラリも要確認

コンテナイメージでの利用を行う場合にも、コンテナの中に含まれる OSS、OSS のライブラリ等とそれらのライセンスを調査する必要があります。

1.4 ライセンスの記載例

- a. OSS をオブジェクトコード、ロードモジュールで利用する場合には、例えば配布するアプリケーションやプログラム上に、ライセンスの記載を行う場合があります。
下記は、COCOA のアプリケーションの例を示しています。

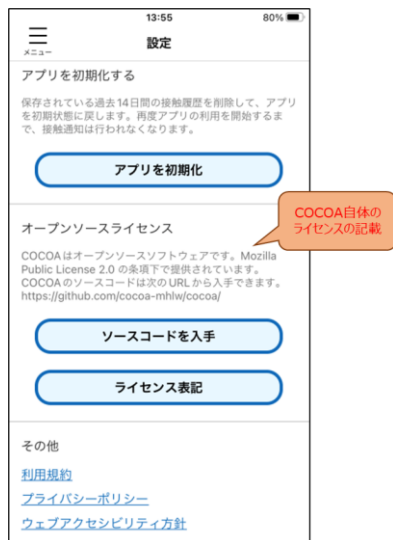


図 3 ライセンスの記載例（アプリ）

- b. OSS をソースコードで利用する場合は、ファイル群の中にライセンスが記載されていることがあります。

下記は、新型コロナウイルス接触確認アプリ「COCOA」の、Web サイト上でライセンスを表示している例です。

（図中水色枠部分、<https://github.com/cocoa-mhlw/cocoa。>）

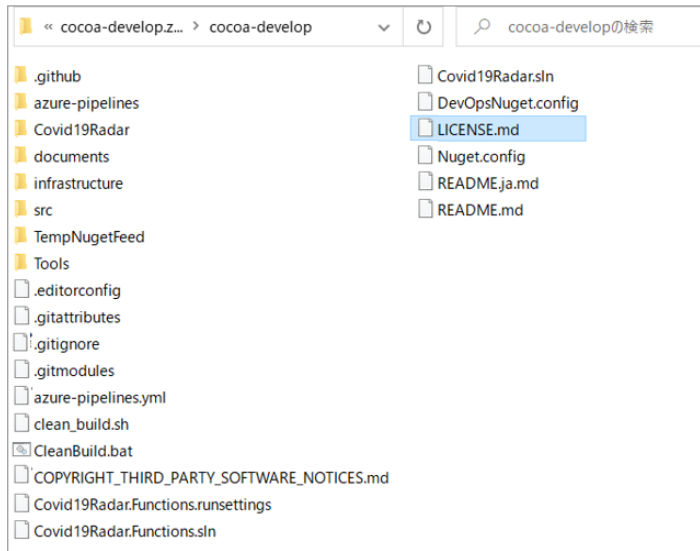


図 4 ライセンスの記載例 (Web サイト)

1.5 複数のライセンスが記載されているケース

OSS の中には、複数のライセンスの中から所望のライセンスを選択して利用できるものがあります。2 つのライセンスから選択可能なものはデュアルライセンス、3 つのライセンスから選択可能なものはトリプルライセンス、またはマルチライセンスと呼ばれています。

- c. デュアルライセンスの例としては Ruby があります。下記 URL を参照ください。
<https://www.ruby-lang.org/en/about/license.txt>



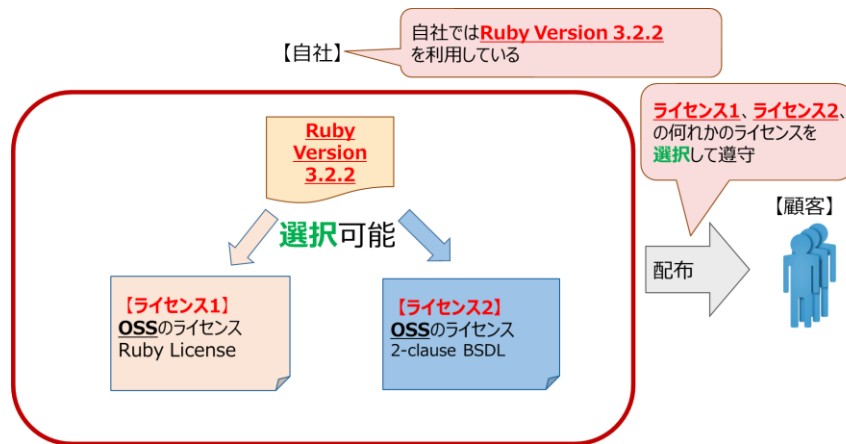


図 5 デュアルライセンスの例

- d. トリプルライセンスの例としては javassist があります。下記 URL を参照ください。
<http://www.javassist.org/>

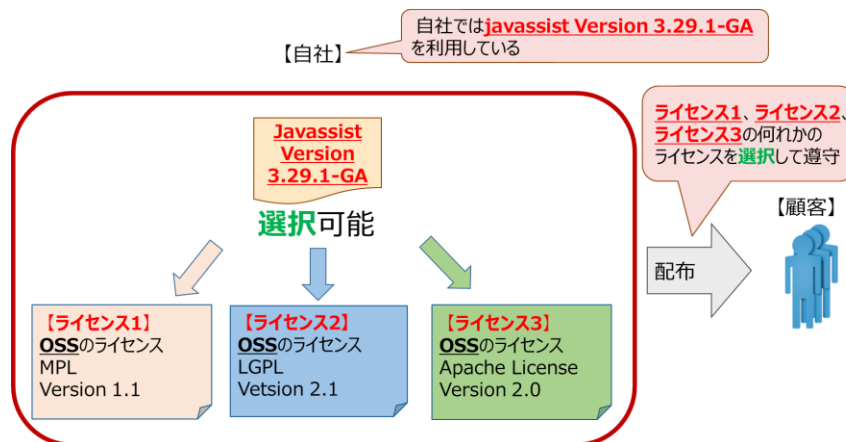


図 6 トリプルライセンスの例

- e. ライセンスの選択については、商用ライセンスと OSS ライセンスから選択する場合があります。
 例えば、OSS のコミュニティで開発されている OSS に対して、ソフトウェア開発ベンダーが、信頼性、可用性等を向上する目的で機能追加し、商用ライセンスで提供する場合があります。この場合、一般的に有償で提供され、ソフトウェア開発ベンダーによるサポートサービスが提供される場合があります。
 また、別の例では、ソース提供義務のある OSS ライセンスと、ソース提供の義務の無い商用ライセンスを選択できるようになっている場合があります。

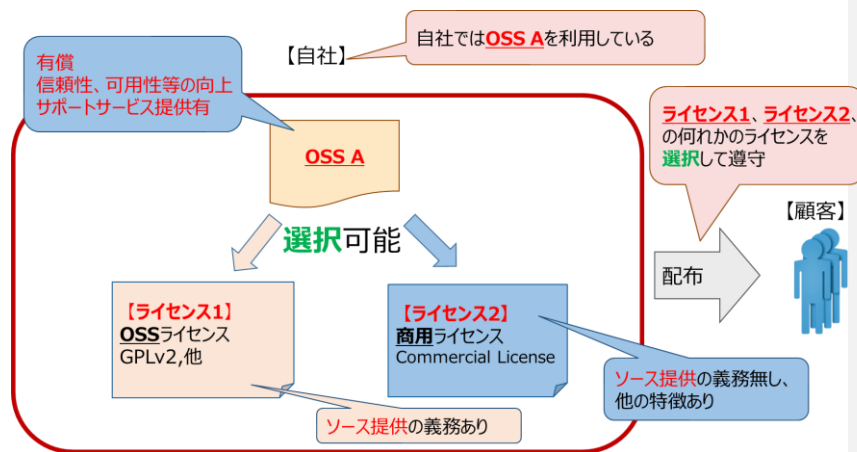


図 7 OSS か商用ライセンスかを選択

1.6 Web 掲載のライセンスで誤認(バージョン確認)

OSS は、そのバージョンによって、ライセンスが変わっている場合があります。インターネット上の Web サイトには最新のバージョンのライセンスが記載されている事が多く、バージョンの古い OSS にも同じライセンスが適用されると判断してしまうと、OSS を配布する場合に、誤ったライセンスで配布してしまうおそれがあります。必ず、配布するバージョンのライセンスを確認しましょう。

- f. 事例 1 : Apache HTTP Server は、Version 1.3.31 および 2.0.49 になる際に、Apache License も 1.1⇒2.0 となりました。

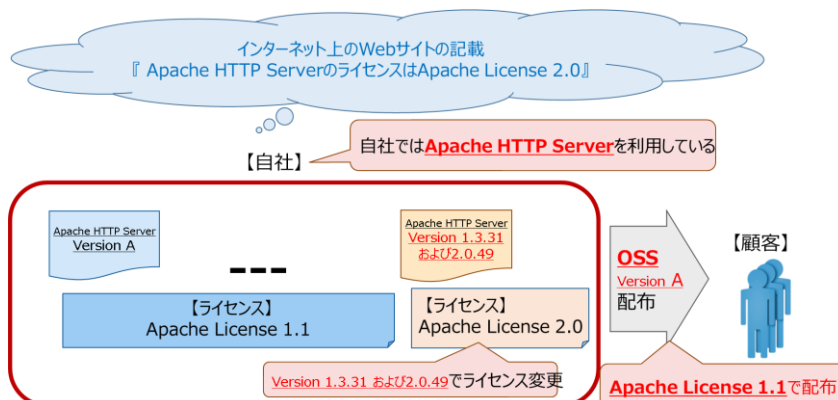


図 8 バージョンが変わるとライセンスも変わる例 (Apache HTTP Server)

g.

- h. 事例 2: jQuery は、Version 1.8 より以前は、GPLv2 と MIT のどちらかを選択するデュアルライセンスでしたが、Version 1.8 で MIT に統一されました。

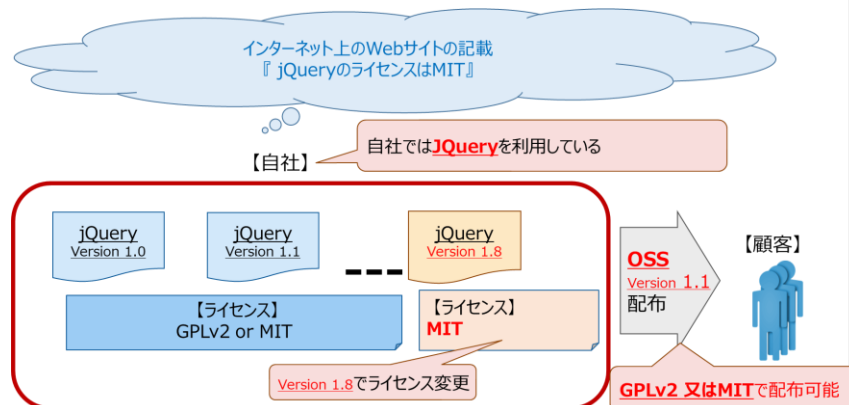


図 9 バージョンが変わるとライセンスも変わる例 (jQuery)

1.7 OSS、ライセンスのリスト化

- i. 1.2～1.6 のように OSS のライセンスを調査して確認することは、OSS を利用する際にそのライセンスの義務を履行するためにも重要になります。また、第 3 章の SBOM を作成するためにも必要になります。
- そこで、ライセンス調査の結果は、下記のような項目についてリストにして纏めておくの良いでしょう。
- (1)OSS 名称
 - (2)OSS のバージョン
 - (3)OSS のダウンロード URL
 - (4)ライセンス名称
 - (5)著作権等の表示
- j. OSS、ライセンスのリスト化を行っておくと、社内で OSS のコンプライアンスについて検討、承認を行う際に、大変有効に活用できます。

2 ライセンス義務の履行

2.1 導入 Q&A

A 社は現在開発中の製品に OSS「X」を搭載することにしました。X はある分野において非常に有名な OSS で、その名前や特徴、開発者について誰でも知っています。そのため A 社では、製品のドキュメントの一部に「OSS「X」を利用しています」と一言だけ添え、その他の詳細な表現は省くことにしました。このときの A 社の対応は OSS ライセンスの遵守作業として果たして十分だったと言えるでしょうか。

残念ながら話はそう単純ではなく、OSS「X」のライセンス条件によってはライセンス違反とみなされる可能性があるため、「十分に遵守された状態」とは言えません。こうした事態を避けるためには、はじめに OSS ライセンスをきちんと理解し、その条件を遵守する必要があります。本章では OSS ライセンスの代表的な遵守作業とそのポイントについて確認します。

2.2 ライセンス遵守作業の概要

- 必須文言（ライセンス文書等）の掲載（2.3 項）
- ソースコードの提供（2.4 項）
- 製品に付帯する契約書（エンドユーザライセンス等）への追記（2.5 項）

また解説の便宜上、ライセンス条件の制約の強さを 5 段階に類型化した「OSS ライセンスのレベル」を用語として取り扱います。各レベルの詳細については「OSS 教育資料、サプライチェーンリスクマネジメント・バージョン ライセンス基礎」第 2 章をご参照ください。

表 1 ライセンス条件の制約の強さ（5レベル）

レベル	ライセンス条件の概要
レベル 1	OSS 配布時、ライセンス文書を添付する
レベル 2	OSS 配布時、レベル 1 に加えて、著作権関連情報等をドキュメントに記載する
レベル 3	OSS 配布時、レベル 2 に加えて、対象 OSS のソースコードを提供する
レベル 4	OSS 配布時、レベル 3 に加えて、対象 OSS と他のソフトウェアを組み合わせると一つの著作物となった全体のソースコードを提供する
レベル 5	レベル 4 に加えて、サービス利用（SaaS/ASP 等）の場合でも、サーバへのアクセス者へ、レベル 4 と同様のソースコードを提供する

2.3 必須文言（ライセンス文書等）の掲載

下記は主な掲載方法の例です。具体的な条件がライセンスに詳述されている場合もあるため、実際に検討する際は各ライセンスの条文をきちんと確認することが重要です。

- 製品の取扱説明書やエンドユーザライセンスなどのドキュメントに含める
- （製品自体に情報表示機能がある場合）オンラインヘルプやオプションメニューなど、ユーザが使用する機能の中で表示する
- （HTML 形式など、ソースコードのまま第三者に提供する場合）ソースコードに予め埋め込まれた著作権表示を削除せずに残しておく

ひとつの注意点として、掲載場所は「OSS の受領者にとって自然に見る場所」で行う必要があります。

例えば「取扱説明書やエンドユーザライセンスが紙の冊子であるにも関わらず、何の案内も無いままに、OSS に関する著作権情報やライセンス条文のみを専用の CD-ROM で保管する」といった方法は、あまり好ましくありません。OSS の受領者が意図的に「製品に含まれる OSS の情報を調べよう」としない限り、目に触れることがないからです。もし他の製品ドキュメントと分離して提示する場合は、その旨を明確に案内する文書を追加するなど、OSS の受領者にとってわかりやすい形となるよう配慮しましょう。

以下にライセンス条文の掲載、謝辞の掲載、広告媒体での謝辞の掲載について説明します。

1) 製品のドキュメントにライセンス条文を掲載する

OSS ライセンスの条文は多くの場合、「COPYING」や「LICENSE」という名前のテキストファイルに格納されています。下記はレベル 1 の OSS ライセンスである MIT LICENSE のテンプレートです。上から順に著作権情報、ライセンス条件、免責条項についてそれぞれ記載されています。

- ライセンス条文の多くは著作権情報やライセンス条件（許諾される行為と制約事項、遵守すべき条件）、免責条項が英文で記載されており、大抵はこれをそのまま OSS 受領者に提供することでライセンス条件を満たすことができます。
- ライセンス条文を掲載する際は**未改変のまま保持**することが重要です。ライセンス条文を変更せず、和訳や解釈を加えることなく、英文原文をそのまま掲載するように留意してください。

Copyright 2023 OpenChain Japan Work Group

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

図 10 レベル 1 の OSS ライセンス (MIT LICENSE)

2) 謝辞を掲載する

レベル 2 の OSS ライセンスの中には謝辞 (Acknowledgements) の掲載を求めるものがあります。多くの場合、謝辞は「このソフトウェアには開発プロジェクト A による成果物が含まれています」という趣旨の一文であり、ライセンス条文にある謝辞の一文をそのまま掲載します。

下記は謝辞の掲載を求めるライセンスと、そのライセンスに基づき実際に謝辞を掲載したソフトウェアの事例です。Libjpeg License は謝辞の一文 ("this software is based in part on the work of the Independent JPEG Group") を製品付帯文書に掲載するよう求めており、それに対して Libjpeg License の OSS を利用する Web ブラウザ「Firefox」では謝辞の一文を "Other Required Notices" に掲載しています。

```
(2) If only executable code is distributed, then the accompanying documentation must state that "this software is based in part on the work of the Independent JPEG Group".
```

図 11 謝辞の掲載を求めるライセンス例

(Libjpeg License, <https://jpegclub.org/reference/libjpeg-license/>)



図 12 謝辞の記載例（Web ブラウザ「Firefox」、about:license#other-notices）

3) 謝辞を広告媒体に掲載する

レベル 2 の一部の OSS ライセンスには、「このソフトウェアの機能または使用について言及するすべての広告媒体にも謝辞を掲載」するよう求める条項があります。例えば OSS を搭載した製品について販売促進を目的とした Web ページ上で OSS の機能について記載する場合、ライセンスの求める謝辞も掲載する必要があります。

下記は広告媒体での謝辞の掲載を求めるライセンスと、そのライセンスに基づき謝辞を広告媒体上に掲載したソフトウェアの例です。自社 Web サイトの製品紹介ページ上で OSS の機能について触れているために、謝辞の一文（“This product includes software developed by the University of California, Berkeley and its contributors.”）を記載しています。

3. All advertising materials mentioning features or use of this software must display the following acknowledgement: This product includes software developed by the University of California, Berkeley and its contributors.

図 13 謝辞を広告媒体へ掲載するよう要求するライセンス条項の例
（BSD 4-Clause Original (Old) License）

* This product includes cryptographic software written by Eric Young (eay@cryptsoft.com)
* This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit.
(http://www.openssl.org/)
* This product includes software developed by the University of California, Berkeley and its contributors.

図 14 通信系組み込み製品における自社 Web サイトの製品紹介ページでの掲載例

2.4 ソースコードの提供

特にソースコードの関連情報として、製品搭載された OSS のバイナリコードを完全に再現するための設定情報やそのバイナリコードを実行するための設定情報も提供するように求めるライセンスもあるため、必ず条文を確認するようにしましょう。

- レベル 3 の OSS ライセンスでは OSS それ自体のソースコードを提供します。OSS を改変した場合は

改変後のソースコードを提供します。

- レベル4のOSSライセンスでは、レベル3の対応に加えて、OSSライセンスの定める他のプログラムのソースコードも提供します。

2.4.1 提供するソースコードの準備

ライセンスの求めるソースコード一式を用意します。ライセンスによっては単にソースコードだけでなく、製品に含めたOSSのバイナリコードを再現し、実行できるようにするために必要となる全ての情報を用意する必要があります。

- 製品搭載されたOSSのバイナリコードを再現、実行するための情報も提供するように求めるライセンスの場合、そのバイナリコードを過不足なく再現できるよう、ソースコードに加えてバイナリコード生成時の情報も用意します。コンパイルオプションや各種設定パラメータ（一般にmakeファイル一式）がこれに相当します。なお、その際に使用したコンパイラなどの「バイナリコードを再現するための環境」それ自体を提供する必要はありません。
- 改変したソースコードを提供する場合はその改変箇所を告示する等、一般的なコーディングルールやマナーに配慮するとよいでしょう。ただしライセンスによっては告示方法に条件を設けているケースもあるため、ライセンスを確認するようにしましょう。
- ソースコードを敢えて紙媒体で配布する等、バイナリコードの再現と実行を意図的に妨げる行為は避けましょう。ライセンス条件に違反する可能性があります。また、例えばOSSにもともと含まれている他人の著作権表示を自社のものに差し替える等、著作権を侵害する行為も避けましょう。

2.4.2 ソースコードの提供方法の選択

OSSライセンスの中には、ソースコードの提供方法について複数の選択肢を提示しているものがあります。そのようなケースでは最も適した提供方法を選択し、準備を進めると良いでしょう。

一方、注意点としてソースコードの提供方法を限定しているOSSライセンスもあります。そのため実際に検討を行う際はライセンス条件を確認のうえ、必要に応じて担当部門（例：法務、知財、OSPOなど）と連携を図るようにしましょう。

本節ではソースコードの提供が必要となるレベル3以上のOSSを製品に適用するケースを例に挙げて解説します。ソースコードの代表的な提供方法は下記の3点です。

- 方法1. 製品に含めるOSSのバイナリコードとともに、必要なソースコード一式も同梱する
- 方法2. 受領者のリクエストに応じてソースコード一式を提供する旨の文書を同梱する
- 方法3. Webサイト上でソースコード一式を提供し、そのURL等を記載した文書も同梱する

方法1. 製品に含めるOSSのバイナリコードとともに、必要なソースコード一式も同梱する

下図のように、両者を製品本体または製品の電子媒体に直接同梱する方法です。

- 製品出荷時にバイナリコードとソースコードを同梱するため、その時点の正確な情報が確実に同梱されるという利点があります。また後述の方法 2、方法 3 と比較して、ソースコードを提供するための体制の構築や維持も不要になります。
- 一方で注意したいのは、製品本体のバージョンアップの際はバージョンアップ版に組み込んだ OSS のバイナリコードと一致したソースコードがきちんと同梱されているか、適宜見直す必要がある点です。また製品本体や製品のドキュメントの容量を圧迫するため、場合によっては媒体を追加する費用がかかるという点もあります。
- 製品リリース前には、必要なソースコード一式が適切に同梱されているか、最終的な出荷物件を確認しましょう。例えば製品出荷に至るまでの一連の工程で、バイナリコードと対応したソースコードを開発部門が下流工程の部門に対してきちんと提供できていないケースや、出荷部門の判断でソースコードが出荷対象から除外されてしまうケースなどが考えられます。同梱漏れにならないよう、各工程の担当部門間で密に連携を図るようにしましょう。

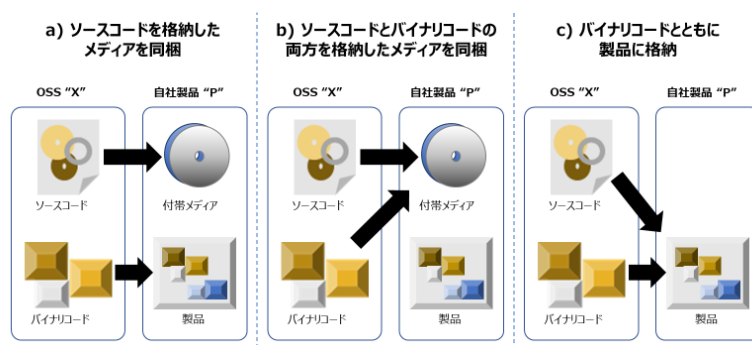


図 15 ソースコードを同梱する三つの形態

方法2. 受領者のリクエストに応じてソースコード一式を提供する旨の文書を同梱する

「受領者のリクエストに応じてソースコードを提供する」旨とその依頼方法（例：問い合わせ窓口）に関する案内を、製品本体または製品ドキュメント（紙媒体や電子媒体）に同梱する方法です。

この場合、案内に基づき受領者が製品開発元に対して「製品搭載されている OSS のソースコードを提供してほしい」と依頼し、それに応じる形でソースコードを提供することになります。

- 受領者からのリクエストがあってはじめて対応する形となるため、コスト面での利点があります。受領者全てがリクエストするとは限らないことに加えて、製品同梱時の媒体コストも削減できるためです。半面、受領者からのリクエストに速やかに対応できる体制を構築し、常に維持していく必要があります。
- 受領者が製品を使用する過程で自然に目を通すものであれば、案内の掲載場所は紙媒体でも電子媒体でもかまいません。製品に付帯する取扱説明書やエンドユーザーライセンスがあればそこに記載してもよいですし、製品が何らかの表示機能を有しているならオンラインヘルプ等

でも問題ありません。

- OSS ライセンスによってはソースコードの提供期間について言及しているケースがあります（例：OSS を含む製品出荷後から少なくとも 3 年間有効など）。
- ソースコードを格納した媒体（例：CD-ROM や DVD など）を提供する際は、送付に必要な実費を請求することも可能です。郵送費や媒体費を鑑み、妥当な価格を設定しましょう。
- 誤ってバージョン違いのソースコード一式を提供してしまわないように、ソースコード一式は製品のバージョンごとに管理しておくとういでしょう。
- 初めから媒体を用意しておく必要はありません。あらかじめ用意しておいてもいいですし、あるいはソースコードの提供依頼があった際に初めて作成しても問題ありません。ただし、提供が遅れるとトラブルに発展することがあるため、速やかに提供しましょう。
- ソースコード一式は、make コマンド等で一括再現可能であれば、tgz 形式や zip 形式などの圧縮形式で提供してもかまいません。
- ユーザからの問い合わせを受ける可能性のある部門（例：ユーザサポート部門）と事前に手順をすり合わせておきましょう。たとえ製品に同梱した案内に窓口を記載していたとしても、思わぬ形で問い合わせを受ける可能性があります。適切な窓口のエスカレートされるよう関係部門にも手順を確認しておくとういでしょう。

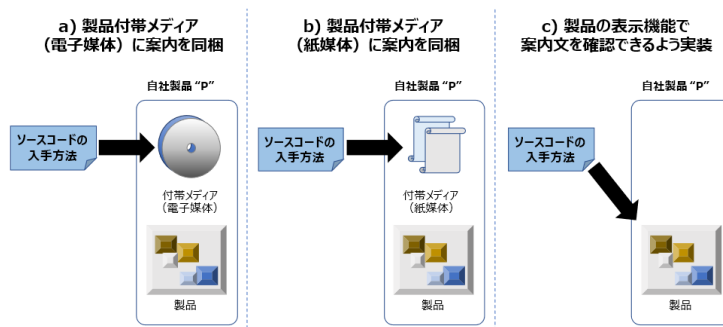


図 16 ソースコード一式を提供する旨の文書を同梱する

方法3. Web サイト上でソースコード一式を提供する旨の文書を同梱する

「Web サイト上でソースコード一式を提供する」旨とその入手方法（例：URL、その他アクセスに必要な情報）に関する案内を、製品本体または製品ドキュメント（紙媒体や電子媒体）に同梱する方法です。

- Web サイト上で受領者が自由にダウンロードできる形となるため、授受に要するタイムラグやコストを低く抑えることが期待できます。一方、当然ながら Web サイトの運用ノウハウを要求されるために、社内の体制構築やインフラ整備等の追加投資が必要となる可能性があります。またダウンロードできない等、何かあった際の連絡先は設けておいた方が良いでしょう。
- 案内の掲載場所は、受領者が製品を使用する過程で自然に目を通すものであれば、紙媒体でも電子媒体でもかまいません。製品に付帯する取扱説明書やエンドユーザライセンスがあ

ればそこに記載してもよいですし、製品の特長として何らかの表示機能を有していればそちらに記載する形でも問題ありません。

- OSS ライセンスによってはソースコードの提供期間について言及しているケースがあります（例：OSS を含む製品出荷後から少なくとも 3 年間有効など）。
- ソースコードの提供窓口は OSS の開発コミュニティではなく、バイナリコードの提供者である製品開発元が担います。仮に受領者に対して、製品開発元の管理が及ばない Web サイトをソースコードの提供窓口として案内してしまった場合、その後の意図しないサイトの閉鎖や移転によって「ソースコードの提供」というライセンス条件を満たせなくなってしまう可能性があるためです。製品搭載したバージョンの OSS を製品開発元できちんと保持し、OSS の受領者に適切に提供できるよう、体制を整えておきましょう。
- ソースコードはフォルダ構造を維持したまま提供しましょう。「バイナリコードの再現と実行を意図的に妨げている」と捉えられないように、過度のファイル分割や公開サイトの分散は避けましょう。
 - Web ブラウザ上でソースコードをそのまま確認できる状態になっている必要はありません。
 - make コマンド等で一括再現可能であれば tgz 形式や zip 形式などの圧縮形式で提供してもかまいません。
- 製品に含まれる OSS を変更した際は、Web サイトで提供しているソースコードも忘れずに更新しましょう。
 - OSS を最新バージョンに差し替えた、あるいは改変を加えた場合は、製品に含まれる OSS と一致させるために Web 上で提供しているソースコードも適宜差し替えましょう。
 - ただし最新バージョンだけではなく旧バージョンの OSS を利用した製品も市場に残るような場合、最新バージョンと同様に旧バージョンのソースコードも引き続き Web 上で提供する必要があります。
- Web サイト上にアクセス制限を特に設けない場合、OSS の受領者以外の誰でもダウンロード可能になります。掲載する情報は必ず確認しましょう。
- 下図は Web サイト上で OSS のソースコードを提供した新型コロナウイルス接触確認アプリ「COCOA」におけるソースコードの提供例です。
 - 「COCOA」は新型コロナウイルスの感染拡大防止を目的として開発された、スマートフォン上で動作可能なアプリケーションです。
 - 「COVID-19Radar」のライセンスに従って、「COCOA」のソースリポジトリ上で「COVID-19Radar」のソースコードを提供しています（図中赤枠部分、<https://github.com/cocoa-mhlw/cocoa>）。

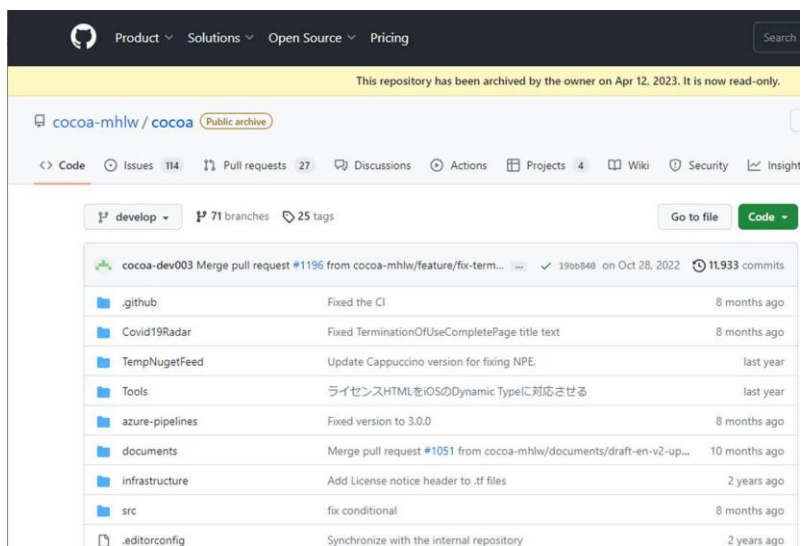


図 17 新型コロナウイルス接触確認アプリ「COCOA」のソースリポジトリ
(Web 上で自身のソースコードを公開)

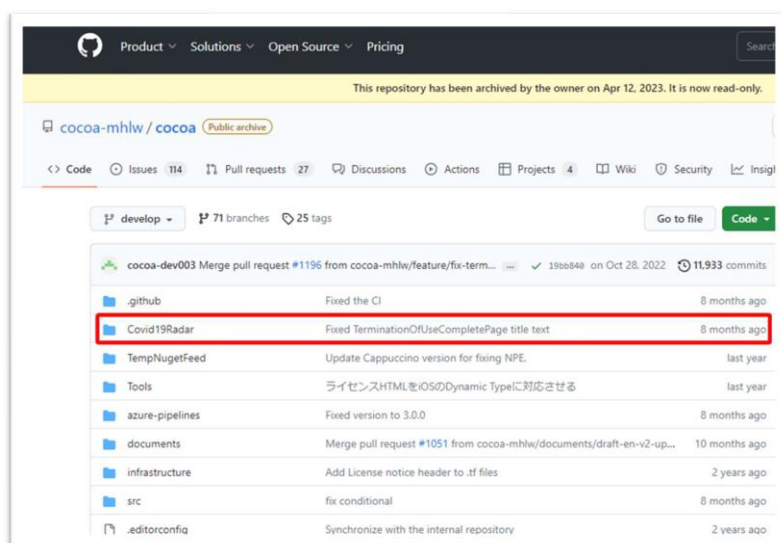


図 18 新型コロナウイルス接触確認アプリ「COCOA」のソースリポジトリ
(赤枠部分にて OSS「COVID-19Radar」のソースコードを公開)

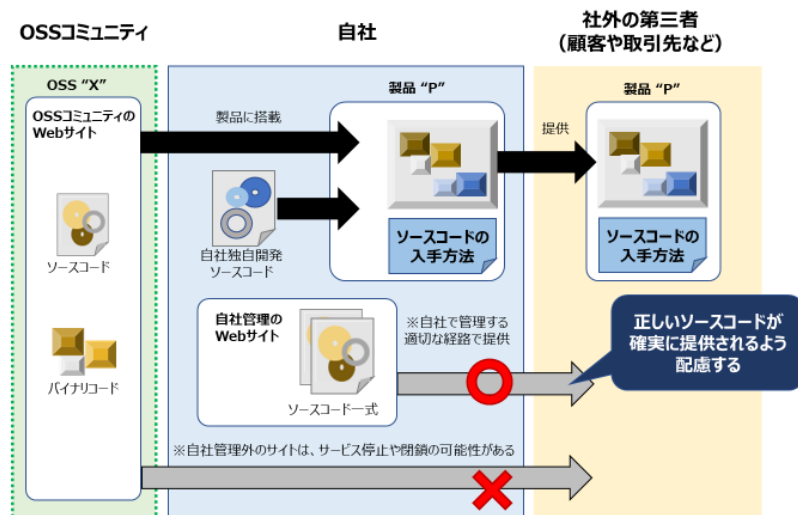


図 19 ソースコードの提供者はバイナリコードの提供者

2.5 製品に付帯する契約書（エンドユーザライセンス等）への追記

しかし OSS を製品に組み込んだ場合、従来のエンドユーザライセンスの条件と OSS ライセンスの条件との間に矛盾が生まれてしまうことがあります。例えば、OSS ライセンスに「OSS ライセンスの条件以外に追加の制限を課してはならない」といった条件や「OSS とリンクしたプログラムのリバースエンジニアリングを許諾せよ」といった条件が含まれるケースがこれに該当します。

こうしたケースでは「OSS については、エンドユーザライセンスの条件よりも、OSS ライセンスの条件が優先する」旨をエンドユーザライセンス側に記載する必要があります。この場合、矛盾する条件については OSS ライセンスの条件が適用され、それ以外の部分についてはエンドユーザライセンスおよび OSS ライセンスの双方の条件が適用されることになります。

そのため OSS を製品に組み込む際は、エンドユーザライセンスの条件が OSS ライセンスの条件に矛盾しないか確認するよう、エンドユーザライセンスの担当部門や法務部門等と連携して、対応を検討することが重要です。

3 SBOM

3.1 ソフトウェアの構成を知る

ここまで、ある OSS に対して対応するライセンスを探し出し、そのライセンスを遵守する方法について説明してきました。ですが、どのような OSS を利用しているかを把握していなければ、対応するライセンスを探し出すことすらできません。なので、どのような OSS を利用しているかを把握することが重要です。また、特に大規模な開発であれば、開発の途中で新しい OSS を追加したり、既存の OSS を入れ替えたりすることもあるでしょう。そのような状況を把握するためには、継続的にしっかりと管理することが必要です。

管理するとなると最初に思い浮かぶのは「一覧表を作成すること」ではないでしょうか。その一覧表のことを SBOM (Software Bill of Materials、ソフトウェア部品表) と呼びます。「一覧表なら (ちょっと面倒かもしれないけど) 難しくない」と思うかもしれませんが、そう簡単ではありません。単なる一覧表ではないのです。組み合わせであるかどうかがライセンス遵守に影響してくる場合がありますので、構成に関する情報も必要になってきます。また、OSS は間接的に他の OSS を利用している場合が多いです。その場合には、直接利用している OSS だけでなく、間接的に利用している OSS についても把握しておく必要があります。

3.2 SBOM を取得/作成する方法

SBOM があれば、利用するソフトウェアのライセンスや著作権などの情報を把握したり、利用するソフトウェアと関連する脆弱性情報を紐づけることが容易にできます。SBOM を取得/作成するには、次の手段が考えられます。取引先や事業環境に応じて適切に手段を選択することが重要です。

- a. ソフトウェアの提供元から入手する：
 - (1) リポジトリ経由で入手した OSS は (自身を除く) SBOM 相当の情報を持っている場合があります。その情報が利用できるかもしれません。
 - (2) ディストリビューター (OSS を一式取り揃えて配布している会社・団体) がサービスの一環として SBOM 相当の情報をまとめているかもしれません。
 - (3) 開発規模がある程度大きくなれば、部分部分に分けてその一部の作成を委託する場合があります。委託した部分に関しては SBOM の作成も含めて委託するのが良いでしょう。
- b. 手作業で解析する：

利用している OSS が少なく、構成もそれほど複雑でなければ、最初から手作業で作ることも可能です。規模がある程度大きくなると次の c 項に述べるようなツールの助けを借りることになるでしょう。
- c. ソフトウェア構成解析のためのツールやソリューションを利用する：

対象となるソフトウェアにどのようなソフトウェアが含まれているのかを調べるツールやソリューションがあります。このようにどのようなソフトウェアが含まれているのかを調べることを「ソフトウェア構成解析 (Software

Composition Analysis: SCA)」と言います。SCA ツールには大きく分けて 2 種類あります。

(1) ソースコードの詳細まで確認するもの。

ソースコード内に書かれたライセンス文や他のソースコードからの部分的な流用も検出できます。一方、自分が書いたソースコードでもたまたま他のソースコードと似てしまった場合にも「流用の可能性あり」と判断されるため、ツールの検出結果を本当に流用なのか、それともたまたま似ているものを誤検出しているのか、人の手で切り分ける必要があります。そのため切り分けまで含めると確認に時間が掛かります。

(2) ファイルやそれより大きな単位で判定するもの。

たまたま他のソースコードに似てしまったものが「流用の可能性あり」と判断されることはありませんので、切り分け作業はほぼ不要です。また判定法の違いにより、上記(1)のツールより判定時間が短い傾向にあります。一方、ソースコードにわずかな変更があっただけでも違うものと判定されるため、実際に該当するソフトウェアを使っているに関わらず、このツールによる検出をすり抜けてしまう場合があります。

OSS をそのまま利用することが多いのか、それとも改変して組み込むことが多いのか、また、開発スピードを優先するのか、それとも時間をかけて開発するのかなど、自社の状況に合わせて適切なツールを選ぶと良いでしょう。最近では両方の特徴をあわせ持ったツールもありますが、どのツールを使う場合でも、自分の使っているツールがどのような機能を持っているのかしっかり把握した上で使うことが重要です。

3.3 SBOM で扱う項目とそのフォーマット

SBOM には自分で作って管理するものという側面以外にも、上で触れたようにソフトウェアの提供元から提供先へ流通するものという側面もあります。円滑な流通のためにはフォーマットの共通化が有効です。そこで米国政府機関の NTIA(National Telecommunications and Information Administration)が「The Minimum Elements For a Software Bill of Materials (SBOM)」という文書を作成しました。そこでは、SBOM に記載すべき項目について(文書の題名の通り)最低限の要素“Minimum Elements”を定義しています。文書中では具体的なフォーマットとして SPDX、CycloneDX、SWID タグの 3 つの名前が挙げられていますが、どのフォーマットも「最低限の要素」を含んでいます。

(a) SPDX :

Linux Foundation 傘下の SPDX プロジェクトにて標準化された SBOM 仕様で、2021 年に ISO で標準化されました(ISO/IEC 5962:2021)。SPDX プロジェクトは「MPL-2.0」などライセンスを短縮して表す SPDX-Identifier を決めていることでも知られています。

(b) CycloneDX :

Web アプリケーションのセキュリティ対策を推進する技術者コミュニティ OWASP (Open Worldwide Application Security Project) で 2017 年に開発されました。

コメントの追加 [1]: 閲覧環境次第かもしれませんが「National Telecommunications and」の間の空白の幅が広いです

コメントの追加 [2]: 以下同文。

コメントの追加 [3]: 閲覧環境次第かもしれませんが空白の幅が広いです

コメントの追加 [4]: 以下同文。

(c) SWID タグ：

アメリカ国立標準技術研究所(NIST: National Institute of Standards and Technology)により標準化された SBOM 仕様で、2015 年に ISO で標準化されました(ISO/IEC 19770-2:2015)。インストールされたソフトウェアの情報を追跡するために設計されています。XML 形式で記述され、サプライヤー名、コンポーネント名、バージョンなどの情報が含まれています。

SPDX のサブセットとして SPDX Lite が仕様化されています (ISO/IEC 5962 及び SPDX v2.3 では Annex.G)。SPDX Lite ではパッケージのレベルで管理すること、Excel などのスプレッドシートで読み書きできるフォーマットとして利用できることを想定し、SPDX のフルセットよりも取り扱う管理情報を絞ったものになっています。取引によっては、SPDX Lite またはこれをカスタマイズしたものを利用することもあるでしょう。

コメントの追加 [5]: 閲覧環境次第かもしれませんが空白の幅が広いです

コメントの追加 [6]: 以下同文と書こうと思ったのですが、これはおそらく「カスタマイズ」を途中で改行したくないのですね、うーん、そういう機能もあるんですね。

3.4 SBOM を管理するためのベストプラクティス

(d) SBOM 管理のポリシーとプロセス：

ISO/IEC 5230 (OpenChain Specification)、とくに§3.3 “Open source content review and approval” に照らして、SBOM の作成と管理に関して、ポリシーとプロセスを定め、文書化し、エビデンスとして管理するようにしましょう。

(e) ソフトウェアライフサイクルに応じた更新および管理：

ソフトウェアの構成に変更がある時、都度、SBOM を更新するようにしましょう。たとえば、ソースコードの状態、ビルド用に準備を整えている状態、ビルドしたバイナリの状態、リリースして運用後にソフトウェア更新をした場合などで、構成が変わることがあります。SBOM は、利用している状態を反映して最新のものになっていることが重要です。

(f) SCA ツール等や構成管理システム：

OSS 自体の大規模化や、間接的に依存する OSS が多い場合などがあります。ツールなどを利用して効率よく SBOM を作成しましょう。ソフトウェア開発環境によっては CI/CD と連携させることも検討すると良いでしょう。また、SBOM はいつでも確認できるように、構成管理システムを用意するなど、集約しておくとう良いでしょう。

(g) e-ラーニング、トレーニング：

ルール、プロセス、ツール等の使い方、エビデンスの管理など、開発者及び SBOM 管理の運用者にトレーニングを実施しましょう。SBOM 管理が当たり前の状態となることで、コンプライアンスや脆弱性に係る問題発生予防や、問題発生時の速やかな対処に繋がります。

(h) 法的要請への対応、業界や取引先への対応、SPDX Lite の利用：

法的な要請により SBOM 及びそれと関連付けて必要となる管理情報が定まる場合があります。また、

業界や取引先によっては、そうした法的要請への対応に加えて独自で必要とする管理情報を設定している場合もあります。できるだけ早めに情報収集し、必要に応じて取引先と協議し、対応を進めるようにしましょう。SPDX Lite が扱う情報で十分の場合もあれば、SPDX Lite にさらに項目を追加して利用する場合もあるでしょう。

(i) オープンソースコミュニティへの参加：

SBOM 管理については、様々なオープンソースコミュニティで情報交換がなされています。SBOM のフォーマットである SPDX や CycloneDX はオープンスタンダードのため、それぞれのオープンソースコミュニティでその仕様策定や管理が議論されています。また、オープンソースコンプライアンスのためのツールを開発するコミュニティでもそうした活動が見られます。さらに、SBOM の相互利用性を検証するために、SBOM やツールのコミュニティが共同し、Plugfest を開催する場合があります。こうしたコミュニティに参加することで、共通の課題の発見や、コミュニティと共に課題解決に取り組むことができます。

3.5 参考資料

- The Minimum Elements For a Software Bill of Materials (SBOM)
米国政府機関 NTIA が出している文書。SBOM というからには最低これだけは記載して欲しいという項目が列挙されています。
https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf
- 2025 Minimum Elements for a Software Bill of Materials (SBOM)
米国政府機関 CISA が出している文書。上記の NTIA が出した The Minimum Elements For a Software Bill of Materials (SBOM) を元にして最新の動向を反映したものになるはずですが、米国政府の閉鎖などの影響を受けて、パブリックコメントの受け付け中で止まっています。
https://www.cisa.gov/sites/default/files/2025-08/2025_CISA_SBOM_Minimum_Elements.pdf
- 米国大統領令（EO）14028
「連邦政府のサイバーセキュリティの強化」に関する米国大統領令。この中で SBOM への言及があり、SBOM が注目されるきっかけとなった文書です。
<https://www.nist.gov/itl/executive-order-14028-improving-nations-cybersecurity>
- EU Cyber Resilience Act (CRA)
欧州サイバーレジリエンス法。セキュリティ対策の一環としてセキュリティリスクを文書として管理することを求めており、ここに SBOM が関係してくる考えられています。
<https://digital-strategy.ec.europa.eu/en/library/cyber-resilience-act>
- オープンソースソフトウェアの利活用及びそのセキュリティ確保に向けた管理手法（2022 年 5 月拡充版）
経済産業省が出している文書。SBOM そのものに関する文書ではなく、「オープンソースソフトウェア

コメントの追加 [7]: 古い？

コメントの追加 [8]: CISA が更新したということを言おうとされてますか。そうであれば、CISA の更新はこちらを上書きするものではない（どちらも有効）ということだそうなので、これはそのまま残しておこうと思います。

が、そうすると CISA 版への言及も必要ですね。

コメントの追加 [9]: Error404 になります

コメントの追加 [10]: とりあえず前の大統領の責任にする現大統領のせいですね。さてどうしましょうか、...

コメントの追加 [11]: ホワイトハウスではなく NIST のものに差し替えました。

（OSS）を利活用するに当たって留意すべきポイントを整理し、そのポイントごとに参考となる取組を実施している企業の事例等を取りまとめた」ものですが、随所に SBOM への言及があります。

<https://www.meti.go.jp/press/2022/05/20220510001/20220510001.html>

- ソフトウェア管理に向けた SBOM（Software Bill of Materials）の導入に関する手引
同じく経済産業省が出している文書。こちらは SBOM そのものに関する文書です。

<https://www.meti.go.jp/press/2023/07/20230728004/20230728004.html>

4 OSS のセキュリティリスクへの対応

4.1 はじめに

現代のソフトウェア開発において重要な役割を果たす OSS ですが、ライセンスに対する理解に加え、セキュリティに対しても正しい理解をする必要があります。2021 年には脆弱性が発見されたログライブラリ「[Apache Log4j](#)」が社会的な問題となりました。また Node.js (NPM) および Python (PyPI) の [パッケージレジストリを標的とした攻撃も報告](#)されています。OSS の脆弱性を悪用した攻撃は増加しており、こうした背景から、企業や組織は今まで以上に OSS のセキュリティ対策が求められてきています。

OSS には同じような機能を持つものや、同じコンポーネントの異なるバージョンによって、脆弱性の数や深刻度、脆弱性の修正速度など、異なるセキュリティレベルが存在することがあります。このため、企業は使用目的に応じた適切な OSS コンポーネントとバージョンを選択する必要があります。また、OSS には元々同じコンポーネントだったものが、複数のパブリッシャーから提供されることで、セキュリティレベルが異なってくる場合があります。脆弱性は常に発見・報告される可能性があるため、最新情報を収集し、適切な対応策を講じる必要があります。

本章では OSS のセキュリティリスクについて分析し、リスクに対する有効な対策を考えます。また OSS のセキュリティリスクへの理解を深めるためのレポートや有用なガイドラインも紹介します。想定読者は、開発者やシステム管理者、セキュリティ担当者、OSS 推進担当者などです。OSS のセキュリティの重要性を再確認し、リスクに対処するためのヒントにいただけると幸いです。

コメントの追加 [12]: 閲覧環境次第かもしれませんが空白の幅が広いです

コメントの追加 [13]: 以下同文、ということ。

4.2 OSS のセキュリティリスクの種類

OSS には多くの人々が開発に携わっていますが、その質は様々です。扱うにあたってどのようなセキュリティのリスクがあるか、考えてみましょう。

a. 脆弱性のあるバージョンの採用：

過去にリリースされた古いバージョンの OSS にはセキュリティの問題がある可能性があります。新しい脆弱性が発見された場合、OSS プロジェクトは通常、脆弱性を修正した最新バージョンをリリースします。古いバージョンの OSS を使用する場合、攻撃者は修正されていない脆弱性を悪用して攻撃してきます。さらに、古いバージョンは、脆弱なプロトコルを使用している場合があり、この点においても最新のセキュリティ基準に適合しないことがあります。

b. パッケージ・ライブラリの脆弱性：

OSS には、それらが依存するパッケージやライブラリがある場合があります。依存関係のあるソフトウェアのバージョンが古く、脆弱性がある場合は a. で挙げたリスクと同様にアプリケーションやシステムに影響を与える可能性があります。

c. パッチの未適用：

OSS プロジェクトが脆弱性に対する修正パッチをリリースした場合、Linux ディストリビューションなどのリポジトリ管理者は OSS プロジェクトがリリースした修正パッチをリポジトリへ配信します。システムの管理者は配信された修正パッチを適用しなければ、脆弱性の脅威に晒され続けます。

d. ソースコードが公開されているためのリスク：

OSS ではソースコードが公開されているため、悪意のある者がソースコードを読んで脆弱性を探し出し、それを悪用する可能性があることに注意が必要です。しかし、ソースコードが公開されているがゆえに、OSS は多数の人々によって監視され、改善されることが多いです。そのため、協力的なコミュニティが存在すれば、そのリスクを軽減できます。この点はメリットとリスクが表裏一体となっています。公開されているコードはユーザが多いほど脆弱性が早期に発見される可能性が高くなり、また修正される可能性も高くなる傾向があります。

e. 開発プロジェクトのサポート：

OSS は開発プロジェクトによって開発が行われるため、バグや脆弱性の修正対応は開発プロジェクトが行います。また、ユーザコミュニティや企業などが脆弱性の対応支援を行うこともあり、第三者から修正プログラムが提供される場合があります。しかし、活発なコミュニティでない場合や、セキュリティリスクの解決を優先しないコミュニティの場合は、問題の解決が遅れる可能性があり、プロプライエタリ・ソフトウェアよりも修正に要する平均時間が高いと報告するレポート(*)もあります。

(*)State of Open Source Security 2022. <https://snky.io/reports/open-source-security/>

OSS プロジェクトの脆弱性修正にかかる時間が増加したとの調査結果. <https://www.conversion.co.jp/tecblog/20220805>

OSS は上述したリスクや傾向からプロプライエタリ・ソフトウェアなど非オープンソースソフトウェアと比較されることがありますが、どちらが安全か決めることはできません。どのようなソフトウェアもセキュリティに対するリスクを持ちますが、リスクを正しく評価し、適切な対策を講じることが重要です。

4.3 OSS のセキュリティリスクを発見する方法

既知の脆弱性を発見する方法として、以下の手段があります。これらの方法を組み合わせ、組織やプロジェクトの状況に応じたベストな脆弱性の収集方法を決定しましょう。

a. セキュリティリスク情報データベース：

CVE や JVN など情報セキュリティ情報を発信するデータベースには、OSS の脆弱性を含む多くのセキュリ

テリリスク情報が掲載されています。

参照できる関連情報公開元

- (1) CVE(Common Vulnerabilities and Exposures):<https://www.cve.org/>
- (2) IPA (情報処理推進機構)脆弱性対策 HP:<http://www.ipa.go.jp/security/vuln/>
- (3) JVN(Japan Vulnerability Notes) 脆弱性対策情報データベース:<http://jvndb.jvn.jp/index.html>
- (4) JPCERT(Japan Computer Emergency Response Center)
CC:<https://www.jpcert.or.jp/vh/top.html>
- (5) OSV(Open Source Vulnerabilities):<https://osv.dev/>
- (6) Known Exploited Vulnerabilities catalog: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

- b. セキュリティ情報配信サービスの利用 :
セキュリティベンダーが発見した脆弱性情報を公開するサービスを確認することで、既知の脆弱性を知ることができます。サービスによっては付加価値として追加の脆弱性対策が案内される場合があります。
- c. セキュリティグループコミュニティへの参加 :
企業や法人からなるセキュリティグループへの参加を通じて、OSS のセキュリティに関する動向、攻撃手口、最新の情報を入手し、関連業種の他会員と解決策とともに模索できます。([OpenSSF](#) や [ICT-ISAC](#) など)
- d. ツールを用いた脆弱性スキャン :
セキュリティベンダーが提供するリスク解析ツールや、OSS として公開されているツールを用いてスキャンすることで、既知の脆弱性を発見できます。

4.4 発見された脆弱性への対応

すべての脆弱性への対応を行うことが難しい場合は、公開されている CVSS(※)の評価結果を用いて脆弱性のシステムへの影響度合いをはかり、環境への影響を踏まえたトリアージが有効です。トリアージとは組織の顧客、経営層、開発者などのステークホルダーとコミュニケーションのうえ、対応する脆弱性の優先度付けを行うことです。早急な対応が必要とされる緊急時は指摘されたすべてのリスクへの対応が現実的に困難な場合があります、許容できないリスクを明確化することが必要です。

なお修正パッチの適用が難しい場合や、パッチ自体がリリースがされない場合は、影響のある機能を一時的に停止する対応や、セキュリティに問題のない代替となるライブラリへの置き換えを検討する必要があります。

※CVSS スコアは、脆弱性の深刻度を 0~10.0 の範囲で評価した指標です。

IPA 共通脆弱性評価システム CVSS 概説. <https://www.ipa.go.jp/security/vuln/CVSS.html>

IPA 共通脆弱性評価システム CVSS 概説によれば、基本評価基準に加え、脆弱性が現在利用可能かど

うかを評価するために使用される「現状評価基準」、脆弱性が組織の環境にどの程度影響を及ぼすかを評価するために使用される「環境評価基準」の要素を活用することで組織に応じた効率的な脆弱性対応の優先度を決定することができます。指標の例として、Access Vector は攻撃がネットワーク経由で行われるか、ローカルからの攻撃が可能かを示す指標です。これにより、攻撃可能範囲や特権が必要かなど、様々な脅威を確認することができます。脆弱性の脅威を正しく理解するためには、CVSS 総合スコアにだけ注目せず各要素も考慮することが重要です。

ただし、CVSS スコアが高くなくても、すでにその脆弱性に対して攻撃が行われている場合には、早急に対応を検討する必要があります。この場合、CISA（米国の社会基盤安全保障庁）が公開している『[Known Exploited Vulnerabilities Catalog \(KEV\)](#)』を参考にできます。このサイトには、既に悪用が観測されている危険な脆弱性が公開されており、こちらも参考にすることが有効です。（米国政府機関の使用システムは Due Date までにパッチを当てる義務があります。）

以上は脆弱性が自社以外のところで発見された場合の対応です。一方、自社の調査や開発過程で、まだ世の中に知られていない（または報告されていない）脆弱性を発見した場合にはどう対処したら良いでしょうか。その場合には、速やかに OSS の開発者やメンテナーに報告することが大切です。報告先は OSS プロジェクトの Web サイトやリポジトリに書かれています。また、同じところに報告に関するポリシーや手順が掲載されている場合があります。脆弱性を発見して報告する場合には、各 OSS プロジェクトの報告に対する方針を確認しましょう。

報告先が不明な場合は、[ZeroDay Initiative](#) へ報告できます。ZeroDay Initiative は Trend Micro が運営する脆弱性情報の収集・報告・研究を行うプログラムです。ZeroDay Initiative では、報告された情報を基に、脆弱性を修正するための情報をベンダーに提供します。そしてベンダーが脆弱性に対応したパッチを公開するという流れになります。

4.5 OSS のセキュリティリスク対策・予防

a. 正規のパッケージの利用：

OSS は公式のリポジトリやパッケージマネージャーからインストールすることを推奨します。悪意のあるパッケージ登録者がいる場合、ダウンロードしたパッケージには悪意のあるアルゴリズムが内包されている可能性があります。公式の認証がされているパッケージディストリビューターからインストールすることで、安全性を確保できます。

b. 環境のバージョンの最新化：

使用する OSS コンポーネントを定期的に最新バージョンへ置き換え、セキュリティアップデートを適用しましょう。

c. サポートライフサイクルの理解：

採用する OSS のサポートライフサイクルを理解することでソフトウェアの適切なバージョン選択、関連パッケージの採用、セキュリティアップデートの計画ができ、OSS の選択に関わるリスクを軽減することができます。

ます。サポートは最新のメジャーバージョンから過去数バージョンが対象となることがありますが、基本的には最新バージョンの採用が推奨されます。OSS プロジェクトによっては、サポートポリシーやソフトウェアライフサイクルは公式 Web ページに記載していることがあるため、これらの情報を確認してみましょう。

d. 脆弱性スキャンツールを用いた定期的な脆弱性のスキャン：

セキュリティベンダーが提供するリスク解析ツールや、OSS として公開されているツールを用いてスキャンを活用して、脆弱性を発見することができます。脆弱性を把握、管理することで、OSS のセキュリティリスクの軽減ができます。

e. 脆弱性を管理する組織の設立とガイド、プロセスの策定：

OSS を使用する企業や組織は、脆弱性を管理する組織を設立し、使用するソフトウェアに対するセキュリティポリシーやガイドライン、対応プロセスを策定し、運用することでリスクを管理できます。

組織やプロセスの設計については [PSIRT Services Framework](#)、[PSIRT Maturity Document](#) や [OpenChain Security Assurance Specification](#) が非常に参考になります。PSIRT Services Framework と PSIRT Maturity Document は JPCERT/CC から [日本語版](#) が公開されています。このフレームワーク、モデルは OSS に限定されませんが、OSS を使った製品にも適用できます。

f. [e-ラーニング、トレーニング](#)：

OSS を使用する開発者や運用者に対して、セキュリティトレーニングを実施することで、OSS の採用に際しての正しいリスクの理解、脆弱性発見後の早期に対処する能力を養うことができます。

g. SBOM の活用：

改めて簡単に言うと、SBOM とはソフトウェアのコンポーネント、バージョン、OSS ライセンス、脆弱性などの情報を記載したドキュメントのことを指します。（詳細は第 3 章で説明・解説がされています。）SBOM を活用することで、前述したコンポーネントの特定工数を短縮でき、脆弱性など OSS セキュリティリスクを軽減することが期待されています。

SBOM は OSS ライセンスに関する問題に対しても有効です。SBOM は Machine Readable なファイル形式であり、一般的に使用するためには、SBOM を読み取り・分析するためのツールが必要になります。

コメントの追加 [14]: 閲覧環境次第かもしれませんが空白の幅が広いです

コメントの追加 [15]: (メモ帳などにコピーしてもらえると分かりますが) 半角空白が一つ入っているだけなので、書き方自体には問題はないです。Google ドライブ側が親切で次の行の先頭にある英単語が途中で改行されないよう上のものを無理やり広げて表示している、ということになります。
(本当の本だとも嫌って文章を見直したりするんですけどね。)

コメントの追加 [16]: 別資料を確認していませんが、OSS 採用する前のおすすめチェックポイントなど紹介ありましたっけ？

4.6 OSS セキュリティに役立つリソース（動向、レポート、ケーススタディ、オンラインコミュニティ、トレーニングなど）

a. [OpenSSF 2022 アニュアル レポート](#)

OpenSSF (Open Source Security Foundation) が、2022 年に公表した報告書です。この

レポートは、OSS セキュリティ、コミュニティの動向、OpenSSF が実施する取り組みなどが掲載されています。

b. [Synopsys オープンソース・セキュリティ&リスク分析レポート\(OSSRA\)](#)

Synopsys 社が公開している OSS のリスク状況を分析したレポートです。M&A に際して行われた監査の情報が基となっており、OSS のセキュリティだけではなく、ライセンスコンプライアンスの問題にも焦点が当てられています。

c. [Snyk オープンソースソフトウェアにおけるセキュリティの現状 \(The State of Open Source Security\)](#)

Snyk 社が公開しているオープンソースセキュリティの状況を分析するレポートです。オープンソースプロジェクトの脆弱性、セキュリティトレンドなど OSS のセキュリティに関する課題についての情報を提供しています。

d. [オープンソースソフトウェアの利活用及びそのセキュリティ確保に向けた管理手法 \(2022 年 5 月拡充版\)](#)

経済産業省が策定したドキュメントで、OSS を利用する企業・組織が、セキュリティリスクを軽減しながら OSS を活用する事例を事例集として提供しています。各企業が OSS を選定・評価する際のポイント、OSS の導入・運用・保守における適切な手順、また OSS のライセンスに関する考慮事項などが説明されています。

e. [OpenChain Security Assurance Specification](#)

OpenChain が定義する、サプライチェーンにおけるソフトウェア品質、セキュリティ、およびコンプライアンス確保のための標準です。この標準は企業がリスク管理の効率化と信頼性の高いソフトウェア提供を目指すことを支援し、コンプライアンス追跡の簡素化や品質向上にも寄与します。

f. [CIS Software Supply Chain Security Guide](#)

米国の非営利組織である CIS (Center for Internet Security) が作成したガイドです。ソフトウェアサプライチェーンのセキュリティに関するベストプラクティスを提供することを目的としています。このガイドは、ソフトウェアサプライチェーンにおけるセキュリティの重要性に焦点を当て、ソフトウェア開発者、サプライヤー、顧客などの関係者に向けた実践的なアドバイス、また内容としてはサプライチェーン攻撃のリスクを軽減するための戦略や、セキュリティのためのフレームワーク、技術的なガイダンスが含まれています。

g. [SLSA \(Supply chain Levels for Software Artifacts\)](#)

SLSA は、ソフトウェアサプライチェーンのセキュリティに関するフレームワークの 1 つです。Google が開発し、2021 年に公開されました。SLSA は、ソフトウェアサプライチェーン内での信頼性を高め、サプライチェーン攻撃から保護するためのベストプラクティスを提供しており、OSS の利用者や開発者が、ソフトウェアサプライチェーンの信頼性を高めるための指針としても利用できます。

・SLSA Whitepaper:

SLSA の概要や目的、各レベルの説明、実装例などが記載されています。

- SLSA Implementation Guide:

SLSA の実装についてのガイドです。SLSA を実装する際に参考にできます。

- SLSA Scorecard:

SLSA の各レベルに対する評価ツールです。自社の OSS の SLSA レベルを評価する際に利用できます。

h. [PSIRT Services Framework](#)、[PSIRT Maturity Document](#)

「4.5 OSS のセキュリティリスク対策・予防」でも紹介した PSIRT Services Framework は、製品のセキュリティインシデントを処理するための PSIRT チームのための指針を提供するフレームワークです。

OSS を活用する製品においても、このフレームワークは役立てられます。一方で PSIRT Maturity Document は企業や組織の PSIRT の構成、プロセスの成熟度を評価する情報が提供されています。これらは JPCERT/CC から日本語版が公開されています。

改訂歴

No.	改訂日 (Version)	改訂内容
1	(V1.0)	